

Latent Energy Action Planning with World Models

Phu Pham and Aniket Bera

Abstract—Latent world models support efficient model predictive control from high-dimensional observations, yet optimizing a single learned latent objective can favor action sequences whose decoder-predicted terminal descriptor does not match the goal descriptor. We introduce Latent Energy Action Planning (LEAP), which treats the complete action horizon as a differentiable variable and optimizes it through a frozen LeWorldModel (LeWM). LEAP couples terminal latent goal matching with a terminal-window state energy. Low energy requires the predicted terminal latent to agree with the goal latent and the decoder-predicted terminal descriptor to agree with the goal descriptor. A frozen goal-conditioned proposal initializes the search, a quasi-Newton solver refines actions through the autoregressive rollout, and post-optimization projection enforces the admissible action range. Across four control domains using the officially released LeWM checkpoints, the complete LEAP planning system raises mean success from 77.5% for LeWM planned with the cross-entropy method (LeWM+CEM) to 94.8% under a matched protocol, a 17.3-percentage-point improvement, while retaining the frozen LeWM representation and predictor.

Index Terms—Deep Learning Methods, Motion and Path Planning, Learning from Demonstration, Model Predictive Control, Robot Learning

I. INTRODUCTION

LATENT world models compress visual observations into predictive states, enabling receding-horizon control without reconstructing pixels. PlaNet [1] and Dreamer [2] established that compact latent dynamics can support online planning and imagined policy learning. More recent systems scale these ideas across diverse domains. DreamerV3 [3] learns behavior through imagined experience with one configuration, while TD-MPC2 [4] performs local trajectory optimization in a learned decoder-free latent model. Joint-embedding predictive architectures replace pixel prediction with feature prediction [5]. I-JEPA [6] and V-JEPA [7] instantiate this principle for images and video. DINO-WM [8] uses pretrained patch features for offline world modeling and test-time goal-conditioned action optimization. LeWorldModel (LeWM) [9] applies joint-embedding prediction to efficient visual control with a small Vision Transformer and autoregressive latent predictor.

Efficiency does not guarantee that the learned planning objective remains reliable under optimization. A planner actively searches for action sequences on which model error is favorable. This search can lower predicted goal cost outside the action and rollout distributions represented by offline data. MOREL [10] and MOPO [11] motivate pessimistic or uncertainty-penalized model-based offline RL, while BEAR [12] identifies instability caused by backing up actions outside the data distribution. Our concern is narrower and occurs inside the planner itself. This *optimization-loop loophole* is distinct from ordinary prediction

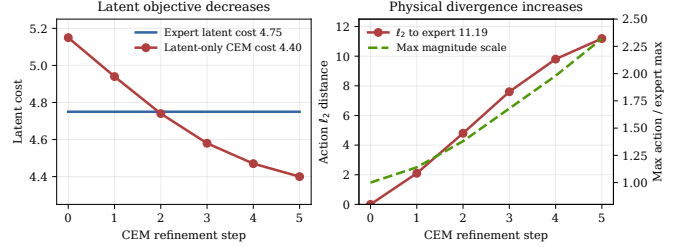


Fig. 1. A representative Push-T latent-objective mismatch. Left: latent-only CEM lowers latent cost below the expert reference. Right: the same refinement increases physical action divergence and raises the maximum action magnitude to $2.32\times$ the expert reference.

error because average predictive accuracy does not establish accuracy on the trajectories selected by optimization.

Figure 1 shows a representative Push-T latent-objective mismatch during cross-entropy method (CEM) [13] refinement. Latent-only CEM lowers the learned latent cost from the expert reference of 4.75 to 4.40. We define magnitude scale as the candidate maximum absolute action divided by the expert-reference maximum absolute action. Over the same refinement process, the ℓ_2 distance to the expert controls reaches 11.19 and the magnitude scale reaches $2.32\times$. The opposing trends illustrate a failure mode of latent-only refinement. The improved latent objective coincides with increasing divergence from the demonstrated controls. This example motivates LEAP’s optimization-aware planner design.

We address terminal ambiguity at planning time without changing LeWM’s representation or predictor. Latent Energy Action Planning (LEAP) treats the complete action horizon as a differentiable variable and backpropagates through the frozen autoregressive rollout. LEAP evaluates the same rollout in LeWM’s latent representation and through a decoder-predicted terminal descriptor. The terminal latent goal cost preserves LeWM’s native visual objective. The terminal-window state cost requires this low latent cost to be corroborated over the final two predictions. A frozen goal-conditioned proposal centers refinement around a data-fitted action prediction, and post-optimization projection prevents inadmissible controls from being selected. These design choices separate predicted terminal-descriptor disambiguation from action feasibility. The loss equation, decoder architecture, normalization rule, terminal window, and energy weights remain shared across domains.

We evaluate action-gradient planning on Push-T, OGBench-Cube, Reacher, and TwoRooms. All comparisons retain the official LeWM encoder and autoregressive predictor while varying the planner. Paired-start ablations isolate the two energy terms. Under the same protocol, LEAP reaches 94.8% mean success, compared with 77.5% for native CEM.

Our contributions are:

- We introduce LEAP, a differentiable planner that opti-

mizes complete action horizons through a frozen latent world model using quasi-Newton refinement and post-optimization action projection.

- We formulate a terminal energy that couples latent-goal matching with decoder-predicted terminal-state matching, requiring agreement across two complementary representations without retraining the world model.
- We conduct an extensive controlled evaluation across four domains and show that LEAP outperforms native LeWM+CEM by 17.3 percentage points while adding only 0.08 s of planning time per trial.

II. RELATED WORK

a) Latent world models.: Latent dynamics reduce the cost of prediction and planning from images. PlaNet [1] plans online in a learned latent model, Dreamer [2] differentiates through imagined trajectories, and DreamerV3 [3] extends imagination-based learning across more than 150 tasks with one configuration. TD-MPC [14] learns a task-oriented latent model for model predictive control. TD-MPC2 [4] subsequently scales local latent trajectory optimization across a broader task suite. LeWM [9] uses joint-embedding prediction and CEM to obtain efficient visual control. LEAP retains LeWM’s learned representation and predictor while redesigning test-time action selection.

b) Joint-embedding prediction.: Joint-embedding predictive architectures (JEPAs) [5] predict target features rather than reconstructing pixels. I-JEPA [6] applies masked feature prediction to images, and V-JEPA [7] extends the principle to video. DINO-WM [8] predicts pretrained DINOv2 patch features from offline trajectories and optimizes action sequences toward visual goals at test time. LeWM [9] instead learns a compact joint-embedding predictor end to end. In both cases the feature space is also the planner’s objective. LEAP preserves this latent objective while adding decoder-predicted terminal-descriptor matching during action optimization.

c) Sampling and gradient planning.: CEM [13] is widely used because it does not require differentiable dynamics. Sampling-based planning with learned models appears in PETS [15] and visual foresight [16]. Differentiable MPC [17] exposes structured controllers to end-to-end gradients through implicit differentiation. Broad search can, however, amplify model error by selecting candidates precisely where the learned objective extrapolates. LEAP instead differentiates a frozen rollout energy directly with respect to the open-loop action tensor and projects the optimized plan into the admissible action range.

d) Offline support and model exploitation.: Offline control methods explicitly address decisions that leave the data distribution. MOPO [11] penalizes uncertain model rollouts, whereas MOREL [10] constructs a pessimistic model for policy optimization. In model-free offline RL, BEAR [12] constrains action selection to reduce bootstrapping error from out-of-distribution actions. LEAP is complementary. It does not fit a policy or value function and does not modify the frozen world model. Instead, it regularizes and projects the action sequence selected by test-time planning.

e) Energy-based control.: Energy-based policies and trajectory objectives provide soft mechanisms for combining complementary goal signals. Implicit Behavioral Cloning [18] models multimodal expert actions with an energy, Composable Energy Policies [19] combine policy factors, and Diffusion Policy [20] performs iterative action refinement through a learned score. LEAP composes latent-goal and terminal-descriptor energies over a multi-step action tensor and differentiates both through a frozen predictive world model.

III. PROBLEM FORMULATION

Let x_t denote the current observation, g a goal observation, and y_g its numerical terminal descriptor. The horizon- H action sequence is $\mathbf{a} = (a_t, \dots, a_{t+H-1}) \in \mathbb{R}^{H \times d_a}$, where d_a is the action-block dimension. A latent world model encodes x_t into $z_t = \Phi_{\text{LeWM}}(x_t)$ and predicts future latents

$$\hat{z}_{t+k} = F_\theta(\hat{z}_{t+k-1}, e(a_{t+k-1})), \quad k = 1, \dots, H, \quad (1)$$

where $\hat{z}_t = z_t$, e is the action-block embedding, F_θ is the frozen autoregressive predictor, and Φ_{LeWM} is the frozen observation encoder. Classical latent planning minimizes a learned task cost

$$\mathbf{a}^\star = \arg \min_{\mathbf{a}} C_{\text{latent}}(\hat{z}_{t+1:t+H}, g). \quad (2)$$

For compactness, subsequent costs are written as functions of $\mathbf{a}$ through the predicted rollout $\hat{\mathbf{z}}(\mathbf{a}) = \hat{z}_{t+1:t+H}$.

A. Motivating Failure of Latent-Only Optimization

Prediction error describes disagreement on sampled inputs. Latent-only planning creates a distinct risk because the optimizer repeatedly concentrates evaluation on action sequences for which learned error is favorable. Optimization can therefore lower latent cost while moving away from the actions and rollouts represented during training. Figure 1 shows this mismatch in a representative Push-T trace. The support analysis below formalizes how this failure can arise. It does not assert that every optimized plan leaves the training distribution.

Let $\mathcal{A}_{\text{data}}$ denote the action-sequence support represented in the offline data, and let $\mathcal{Z}_{\text{data}}$ denote the corresponding latent-rollout support. Minimizing only terminal latent error imposes neither $\mathbf{a} \in \mathcal{A}_{\text{data}}$ nor $\hat{\mathbf{z}}(\mathbf{a}) \in \mathcal{Z}_{\text{data}}$. The planner can therefore select a sequence for which

$$C_{\text{latent}}(\mathbf{a}^\star) < C_{\text{latent}}(\mathbf{a}_{\text{data}}), \quad (3)$$

$$\mathbf{a}^\star \notin \mathcal{A}_{\text{data}} \quad \text{or} \quad \hat{\mathbf{z}}(\mathbf{a}^\star) \notin \mathcal{Z}_{\text{data}}. \quad (4)$$

Here, $\mathbf{a}_{\text{data}} \in \mathcal{A}_{\text{data}}$ is a reference sequence. This condition does not require a particular optimizer. It arises whenever optimization concentrates evaluation on model errors that are rare under the data distribution. Low predicted goal error alone is therefore insufficient evidence that an action plan reaches the intended physical configuration. This motivates complementing the visual latent objective with decoder-predicted terminal-descriptor matching and enforcing admissible controls through post-optimization action projection.

B. Projected Energy-Based Planning Objective

LEAP augments the terminal latent objective with terminal-state matching. Given K proposal-seeded initializations $\{\mathbf{a}_0^{(k)}\}_{k=1}^K$, it refines and then projects each candidate before selection:

$$\begin{aligned}\tilde{\mathbf{a}}^{(k)} &= \text{LBFGS}(E_{\text{total}}, \mathbf{a}_0^{(k)}), \\ \mathbf{a}^{(k)} &= \Pi_{\mathcal{A}}(\tilde{\mathbf{a}}^{(k)}), \quad k^* = \arg \min_k E_{\text{total}}(\mathbf{a}^{(k)}).\end{aligned}\quad (5)$$

Here, LBFGS denotes the L-BFGS solve, $\mathcal{A}$ is the normalized environment action domain, and $\Pi_{\mathcal{A}}$ denotes Euclidean projection onto that domain. The selected plan is $\mathbf{a}^* = \mathbf{a}^{(k^*)}$. Let D_ψ be a frozen decoder from LeWM latent space to a normalized state descriptor. LEAP uses

$$E_{\text{total}}(\mathbf{a}) = C_{\text{latent}}(\mathbf{a}) + \lambda_s E_{\text{terminal}}(\mathbf{a}), \quad (6)$$

where $\lambda_s > 0$ is the terminal-state weight. This energy combines the native visual goal with decoder-predicted terminal-descriptor matching.

Both costs are nonnegative. Thus, for any $\epsilon \geq 0$, $E_{\text{total}}(\mathbf{a}) \leq \epsilon$ implies $C_{\text{latent}}(\mathbf{a}) \leq \epsilon$ and $E_{\text{terminal}}(\mathbf{a}) \leq \epsilon/\lambda_s$. Because neither term is negative, a low total cannot mask a large disagreement in either representation.

The two terms address predicted terminal-descriptor ambiguity through complementary goal representations. The latent goal cost asks whether the terminal prediction resembles the goal in LeWM’s representation. The terminal-state cost asks whether the decoder-predicted terminal descriptor matches the goal descriptor under a normalized metric. A low latent score must therefore be corroborated by the decoder prediction instead of serving as the planner’s sole evidence of goal attainment. Terminal-state matching does not itself impose membership in $\mathcal{Z}_{\text{data}}$. Neither energy term constrains action magnitude. Post-optimization projection separately enforces admissible controls at the environment interface. The resulting objective combines terminal disambiguation with action feasibility without treating either as a certificate of offline-data support.

IV. METHODOLOGY

A. Differentiable Native-LeWM Rollout

LEAP changes action selection while leaving LeWorldModel (LeWM) unchanged. The native visual encoder maps the current image and goal image to latent vectors,

$$z_t = \Phi_{\text{LeWM}}(x_t), \quad z_g = \Phi_{\text{LeWM}}(g), \quad (7)$$

and the frozen autoregressive predictor rolls forward under a candidate action sequence,

$$\hat{z}_{t+k} = F_\theta(\hat{z}_{t+k-1}, e(a_{t+k-1})), \quad k = 1, \dots, H. \quad (8)$$

The encoder and predictor parameters remain frozen. Unlike native CEM, LEAP retains the computation graph from the predicted latents to the candidate actions. This graph supplies gradients for updating the entire horizon instead of fitting a sampling distribution to ranked candidates.

Native LeWM+CEM treats the frozen rollout model as a black-box evaluator and updates a sampling distribution

from ranked action sequences. LEAP instead preserves the rollout computation graph and directly refines the complete action horizon using gradients of the combined terminal energy. Because both planners retain the same LeWM representation and dynamics predictor, their distinction lies in how candidate actions are initialized, refined, and constrained. LEAP therefore converts the pretrained world model from a sampling-based scorer into a differentiable action-planning model.

Figure 2 summarizes the planner. Both energy terms act on the same frozen rollout, and their sum supplies a single action gradient.

B. Energy-Based Latent Action Planning

For candidate sequence $\mathbf{a} \in \mathbb{R}^{H \times d_a}$, the final LEAP design minimizes

$$E_{\text{total}}(\mathbf{a}) = C_{\text{latent}}(\mathbf{a}) + \lambda_s E_{\text{terminal}}(\mathbf{a}), \quad \lambda_s = 10. \quad (9)$$

The terminal latent goal cost is LeWM’s native planning objective,

$$C_{\text{latent}}(\mathbf{a}) = \|\hat{z}_{t+H}(\mathbf{a}) - z_g\|_2^2. \quad (10)$$

It supplies semantic goal direction in the representation on which LeWM was trained.

To predict a complementary terminal descriptor, a frozen decoder D_ψ maps each predicted latent to a standardized state descriptor,

$$\hat{y}_{t+k} = D_\psi(\hat{z}_{t+k}). \quad (11)$$

The decoder is fitted from an offline training subset using frozen LeWM embeddings and benchmark state descriptors. Success labels are not used. Each decoder is trained for 50 epochs on 9,000 examples, with 1,000 held-out examples, using the same two-hidden-layer MLP with widths 256–256. Each coordinate is standardized with training-set statistics. The architecture, fitting procedure, coordinate-wise normalization, and loss definition are identical across domains. Only the fitted parameters and dimensionality follow the corresponding frozen checkpoint and descriptor schema.

Let y_g be the standardized goal descriptor and let $L = 2$ denote the terminal window. The shared terminal-window state energy is

$$E_{\text{terminal}}(\mathbf{a}) = \frac{1}{LD} \sum_{k=H-L+1}^H \|D_\psi(\hat{z}_{t+k}(\mathbf{a})) - y_g\|_2^2, \quad L = 2, \quad (12)$$

where D is the descriptor dimension. The combined energy turns goal evaluation into cross-representation terminal agreement. A candidate that enters a low latent-cost region but predicts a terminal descriptor inconsistent with y_g remains penalized. Scoring both final predictions further discourages plans that only cross the goal at a single terminal instant. Equation (9), including $L = 2$ and $\lambda_s = 10$, is unchanged across Push-T, OGBench-Cube, Reacher, and TwoRooms. The algebraic objective contains no domain-specific branches, coordinate masks, or success thresholds.

Fig. 3 provides a geometric analysis of a local 2-D Push-T action slice around an expert action. The asymmetric level sets, secondary basins, and curved descent paths expose the

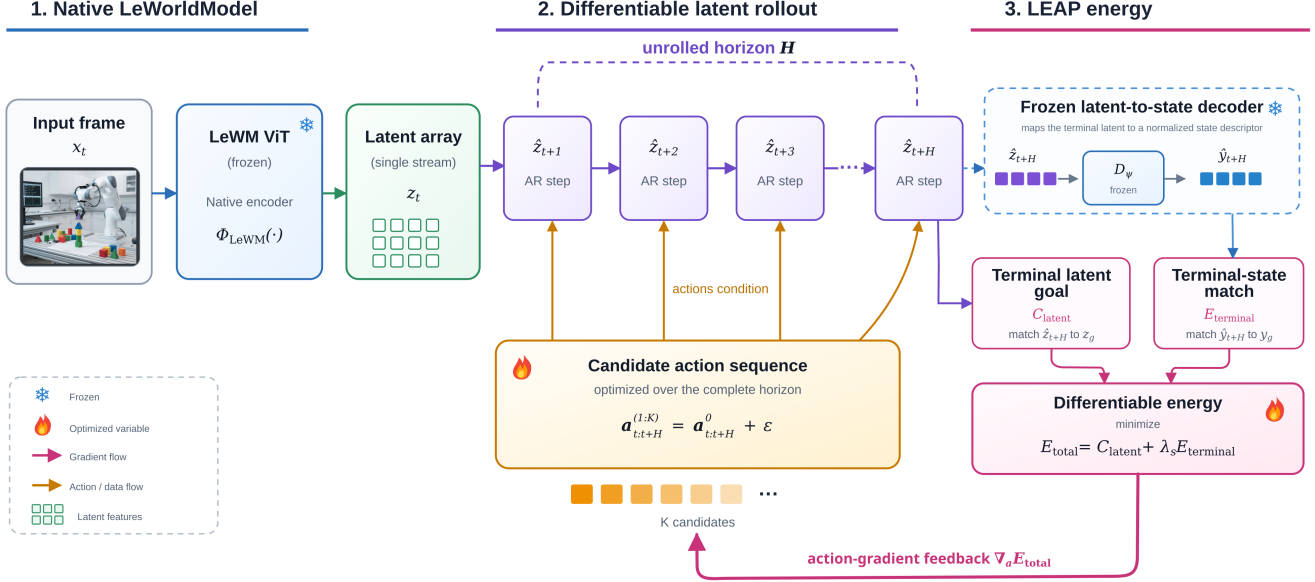


Fig. 2. Overview of LEAP. Frozen LeWM encodes the current observation and unrolls each of K candidate action sequences through its autoregressive latent predictor. Candidates are initialized around a goal-conditioned proposal and optimized over the complete horizon, with each action block conditioning its corresponding rollout step. The terminal latent-goal term compares the predicted and goal latents, while a frozen decoder provides normalized descriptors for terminal-state matching. Their sum forms the total energy, whose gradient propagates through the rollout to refine the candidate actions.

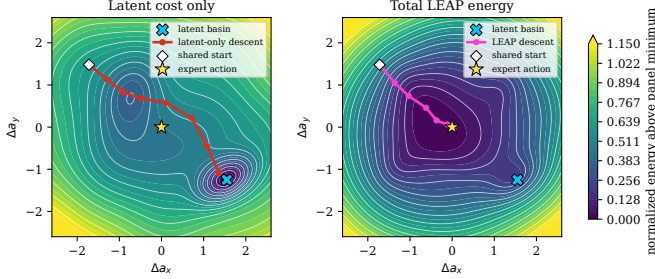


Fig. 3. 2-D Push-T action-slice energy surface around an expert action. Left: C_{latent} contains a misleading off-center basin whose decoder-predicted terminal descriptor is inconsistent with the goal (cyan cross). Right: the total LEAP energy raises this basin and redirects descent toward joint latent and terminal-state agreement (yellow star). Both panels use the same normalized color scale. Irregular local geometry shows that terminal-state matching reshapes rather than simply rescales the action landscape.

nonconvex geometry faced by action optimization. In the left panel, a misleading latent basin has low cost but is inconsistent with the decoder-predicted terminal descriptor. In the right panel, terminal-state matching raises that basin and redirects descent toward cross-representation agreement. The second term therefore changes the preferred action directions rather than merely rescaling the latent objective.

C. Initialization, Optimization, and Projection

LEAP initializes candidates with an auxiliary goal-conditioned action proposal trained on offline trajectories and frozen for evaluation. The proposal is a three-hidden-layer MLP with SiLU activations. It maps benchmark-provided planning context to a normalized five-block action sequence. Training minimizes mean-squared error to the corresponding expert sequence using a 95/5 training/validation split, and we retain the minimum-validation-loss checkpoint.

The proposal supplies only the search center and has zero weight in Eq. (9). Its deterministic output defines the first candidate. When $K > 1$, seeded perturbations around this center provide additional local starts. This separation keeps the optimized energy compact while avoiding a cold start in the nonconvex action landscape.

For each candidate, LEAP computes

$$\nabla_{\mathbf{a}} E_{total} = \frac{\partial E_{total}}{\partial \hat{\mathbf{z}}} \frac{\partial \hat{\mathbf{z}}}{\partial \mathbf{a}}. \quad (13)$$

The final planner uses L-BFGS [21] with learning rate 1.0, ten internal iterations, history size 100, default gradient and parameter-change tolerances of 10^{-7} and 10^{-9} , and strong-Wolfe line search. After the internal solve, post-optimization action projection clamps every candidate to the normalized environment action range. Energy is recomputed on the projected candidates, and selection uses the recomputed values. This enforces admissible controls without changing the internal L-BFGS trajectory.

For Q objective-and-gradient evaluations, K candidate sequences, and rollout horizon H , planning requires $O(QKH)$ predictor steps at fixed model width. L-BFGS line search can invoke more closures than its nominal iteration budget, so we measure synchronized planner time directly rather than treating iteration count as compute. Differentiation retains candidate-specific rollout activations. Memory therefore grows with K , H , and model width.

After selecting the lowest-energy candidate independently for each environment, LEAP executes five action blocks (25 primitive environment steps) before replanning.

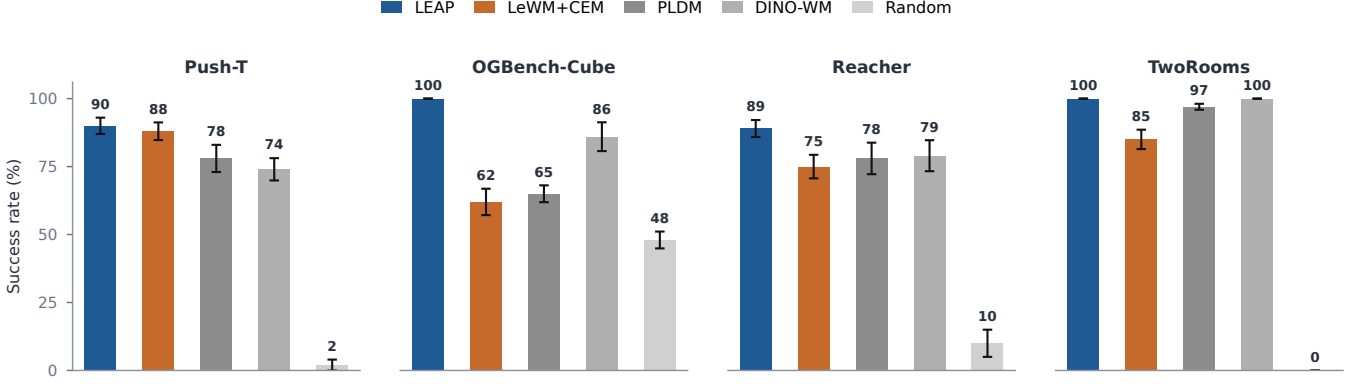


Fig. 4. Task success rates (%). LEAP and LeWM+CEM use our matched controlled 100-trial protocol. Black error bars show one binomial standard error for controlled evaluations and reported variation for contextual baselines.

Algorithm 1: LEAP action-gradient optimization

Input: Observation x_t , goal (g, y_g) , horizon H , starts K , L-BFGS budget M , weight λ_s

Output: Configured receding-horizon prefix

- 1 Encode $z_t \leftarrow \Phi_{\text{LeWM}}(x_t)$ and $z_g \leftarrow \Phi_{\text{LeWM}}(g)$ with native LeWM
- 2 Initialize K candidates from the frozen action proposal and seeded perturbations
- 3 Roll out candidates and compute Eqs. (10) and (12)
- 4 $\tilde{\mathbf{a}}^{(k)} \leftarrow \text{L-BFGS}_M(E_{\text{total}}, \mathbf{a}_0^{(k)})$, $k = 1, \dots, K$
- 5 $\mathbf{a}^{(k)} \leftarrow \Pi_{\mathcal{A}}(\tilde{\mathbf{a}}^{(k)})$
- 6 $s_k \leftarrow E_{\text{total}}(\mathbf{a}^{(k)})$
- 7 Reject the plan if $\{k : s_k < \infty\} = \emptyset$
- 8 $k^* \leftarrow \arg \min_{k: s_k < \infty} s_k$
- 9 **return** $\mathbf{a}_{t:t+4}^{(k^*)}$

V. EXPERIMENTAL EVALUATION

A. Domains and Evaluation Protocol

We evaluate Push-T, OGBench-Cube, Reacher, and TwoRooms using the officially released LeWM checkpoints. The visual encoder and autoregressive predictor remain frozen. Current and goal observations are sampled 25 environment steps apart within the same offline trajectory. Each planner uses a five-transition latent horizon, executes five action blocks (25 primitive environment steps) before replanning, and operates under a 50-step interaction budget. Candidate counts are 32 for Push-T, one for OGBench-Cube, and four for Reacher and TwoRooms. Actions are restricted to $[-2, 2]$, except in TwoRooms, which uses $[-1.6, 1.6]$.

We select the shared terminal window $L = 2$, terminal-state weight $\lambda_s = 10$, and optimizer configuration using the paired 50-trial sweeps. These settings are fixed across domains before the final 100-trial evaluation, which uses a different start schedule. No setting is adjusted per task. We also report the corresponding 50-trial final-system results to assess evaluation stability. The LEAP-CEM comparison uses matched sampled starts, task configurations, interaction budgets, official evaluation code, and checkpoints.

TABLE I

RESULTS UNDER THE 50- AND 100-TRIAL PROTOCOLS. ENTRIES ARE SUCCESS RATE $\pm$ ONE BINOMIAL STANDARD ERROR (%). MEAN IS UNWEIGHTED ACROSS TASKS. BOLD IDENTIFIES LEAP.

Trials	Planner	Push-T	OGBench-Cube	Reacher	TwoRooms	Mean
50	LEAP	94.0 ± 3.4	100.0 ± 0.0	92.0 ± 3.8	100.0 ± 0.0	96.5 ± 1.3
	LeWM+CEM	92.0 ± 3.8	62.0 ± 6.9	72.0 ± 6.3	82.0 ± 5.4	77.0 ± 2.9
100	LEAP	90.0 ± 3.0	100.0 ± 0.0	89.0 ± 3.1	100.0 ± 0.0	94.8 ± 1.1
	LeWM+CEM	88.0 ± 3.2	62.0 ± 4.9	75.0 ± 4.3	85.0 ± 3.6	77.5 ± 2.0

Success is binary. Both planners are timed one environment at a time on the same task-assigned GPU. Planning time includes candidate initialization and action optimization but excludes model loading, environment stepping, and visualization. Budgeted steps per trial average all trials, using the first successful primitive step for successful trials and the full interaction budget for failures.

B. Overall Task Performance

Figure 4 compares LEAP with our controlled recomputation of native LeWM+CEM. We reproduce the PLDM, DINO-WM, and Random rates reported by LeWM [9] for context because we did not rerun those methods. We omit baselines that LeWM did not evaluate on all four tasks, giving every displayed method the same domain coverage.

LEAP reaches 90.0% on Push-T, 100.0% on OGBench-Cube, 89.0% on Reacher, and 100.0% on TwoRooms, yielding a 94.8% mean. Under the same protocol, LeWM+CEM reaches 88.0%, 62.0%, 75.0%, and 85.0%, respectively, for a 77.5% mean. LEAP changes the planner and adds the terminal-state readout while retaining the frozen LeWM representation and predictor. This controlled comparison evaluates complete planning systems and does not isolate the effects of all planner components individually.

Table I reports both protocols. The 50-trial evaluation yields a 96.5% mean for LEAP and 77.0% for LeWM+CEM. LEAP's unweighted mean changes by 1.8 percentage points at 100 trials, and LeWM+CEM changes by 0.5 point. No task-level rate changes by more than four points, leaving every ranking unchanged.

TABLE II

REPORTED SUCCESS IS IN PERCENT. PLANNING TIME AND ENVIRONMENT STEPS ARE PER TRIAL. BOLD IDENTIFIES LEAP.

Planner	Success (%) $\uparrow$	Planning / trial (s) $\downarrow$	Steps / trial $\downarrow$
LEAP	94.8	1.28	21.06
LeWM+CEM	77.5	1.20	26.41

TABLE III

LEAP ENERGY ABLATION. PAIRED 50-TRIAL PROTOCOL. TASK ENTRIES AND MEAN ARE SUCCESS RATES (%). MEAN IS UNWEIGHTED. SUPERSCRIPTS SHOW PERCENTAGE-POINT CHANGES FROM FULL LEAP. BOLD MARKS THE BEST MEAN.

Setting	OGBench-				
	Push-T	Cube	Reacher	TwoRooms	Mean
Full LEAP	94	100	92	100	96.5
Latent goal only	90 ⁻⁴	94 ⁻⁶	86 ⁻⁶	94 ⁻⁶	91.0 ^{-5.5}
Terminal-state only	34 ⁻⁶⁰	56 ⁻⁴⁴	22 ⁻⁷⁰	96 ⁻⁴	52.0 ^{-44.5}

Table II reports matched planning time and environment interaction. Steps per trial average all trials and assign failures the full interaction budget.

LEAP improves mean success by 17.3 percentage points and uses 5.35 fewer budgeted steps per trial. Differentiable L-BFGS refinement adds 0.08 s of planning time per trial (1.28 versus 1.20 s). This modest increase accompanies higher success and fewer interactions.

C. Complementarity of the Two Energies

We compare the complete objective with both single-term variants while holding the checkpoint, proposal, optimizer, candidate count, projection, and paired starts fixed. The latent-goal-only variant differentiates LeWM’s native objective through the same planner, while the terminal-state-only variant removes the visual latent term.

The decomposition reveals an asymmetric complementarity. Latent goal matching is already strong, reaching 90.0% on Push-T, 94.0% on OGBench-Cube, 86.0% on Reacher, and 94.0% on TwoRooms for a 91.0% mean. Terminal-state matching alone averages 52.0%. It is a complementary signal, not a replacement for visual goal identity. Within this paired ablation matrix, their combination is best on every domain and raises the task mean to 96.5%. With optimization and initialization unchanged, the comparison isolates the objective: semantic latent matching supplies broad goal direction, while terminal-state matching resolves residual decoded error and prevents latent similarity from acting as the sole selection criterion.

Table IV compares the first selected plans on paired starts. LEAP lowers mean decoder-predicted terminal-state error in every domain, delivering a 6.7% task-average reduction over latent-goal-only planning. It selects a lower-error plan on 89% of starts. These consistent improvements show that terminal-state matching reliably corrects plans that latent-only optimization ranks favorably despite decoded terminal mismatch. With optimizer and initialization fixed, the comparison isolates this contribution and demonstrates stronger cross-representation goal agreement.

The selected-plan analysis complements task success by showing across domains that the additional energy systematically changes action selection rather than benefiting only a

TABLE IV

FIRST-DECISION CROSS-REPRESENTATION AGREEMENT. RATIOS COMPARE LEAP WITH LATENT-GOAL-ONLY ERROR. PAIRED LOWER DENOTES IMPROVED STARTS. MEAN IS UNWEIGHTED ACROSS TASKS.

Metric	Push-T	OGBench-			Mean
		Cube	Reacher	TwoRooms	
Terminal ratio	0.940	0.917	0.921	0.954	0.933
Paired lower	89%	98%	96%	73%	89%

TABLE V

DAMPING-ENERGY WEIGHT ABLATION WITH $\lambda_s = 0$. PAIRED 50-TRIAL PROTOCOL. TASK ENTRIES AND MEAN ARE SUCCESS RATES (%). MEAN IS UNWEIGHTED. SUPERSCRIPTS SHOW PERCENTAGE-POINT CHANGES FROM LATENT GOAL ONLY. BOLD MARKS THE BEST MEAN.

Setting	OGBench-				
	Push-T	Cube	Reacher	TwoRooms	Mean
$\lambda_v = 0$	90	94	86	94	91.0
$\lambda_v = 0.1$	92 ⁺²	98 ⁺⁴	76 ⁻¹⁰	98 ⁺⁴	91.0
$\lambda_v = 0.3$	94 ⁺⁴	98 ⁺⁴	82 ⁻⁴	98 ⁺⁴	93.0 ⁺²
$\lambda_v = 1$	90	96 ⁺²	84 ⁻²	98 ⁺⁴	92.0 ⁺¹
$\lambda_v = 3$	92⁺²	98⁺⁴	88⁺²	100⁺⁶	94.5^{+3.5}
$\lambda_v = 10$	92 ⁺²	96 ⁺²	78 ⁻⁸	98 ⁺⁴	91.0

small subset of trials. Together with the component ablation, this result identifies cross-representation terminal agreement as the source of the improvement over latent-goal-only planning.

D. Terminal Regularization Ablations

We test whether suppressing decoded motion near the horizon improves latent-goal optimization. For $\hat{y}_{t+k} = D_\psi(\hat{z}_{t+k})$, define

$$E_{\text{damp}}(\mathbf{a}) = \frac{1}{(L-1)D} \sum_{k=H-L+2}^H \|\hat{y}_{t+k} - \hat{y}_{t+k-1}\|_2^2, \quad (14)$$

which penalizes terminal-window descriptor motion. This ablation-only term has coefficient λ_v .

Table V reports the complete four-domain damping-weight sweep with terminal-state matching disabled. The setting $\lambda_v = 3$ is the only tested weight that improves every task over latent goal only and raises the mean from 91.0% to the best damping-only mean of 94.5%. Smaller and larger weights regress Reacher, showing that the penalty requires appropriate scaling. We use $\lambda_v = 3$ in the composition study of Table VII.

Table VI disables damping energy and varies $\lambda_s \in \{0, 0.1, 0.3, 1, 3, 10\}$. Intermediate weights of 1 and 3 reduce mean performance because Reacher drops to 68%, whereas $\lambda_s = 10$ reaches 96.5% and improves every task over latent goal only. Reacher is particularly sensitive because success requires precise terminal end-effector placement: at intermediate weights, comparably influential latent and terminal-state gradients can steer refinement toward different local basins, whereas $\lambda_s = 10$ makes terminal grounding decisive enough to select actions consistent with the decoded goal geometry. This non-monotonic optimization geometry motivates the shared final setting.

Finally, Table VII tests whether the two best auxiliary settings compose. Damping energy alone improves the mean to 94.5%, but adding $\lambda_v = 3$ to the terminal-state term at $\lambda_s = 10$ reduces OGBench-Cube from 100% to 96%, Reacher from 92% to 86%, and the mean from 96.5% to 94.0%. The measured regression

TABLE VI

TERMINAL-STATE WEIGHT ABLATION WITH $\lambda_v = 0$. PAIRED 50-TRIAL PROTOCOL. TASK ENTRIES AND MEAN ARE SUCCESS RATES (%). MEAN IS UNWEIGHTED. SUPERSCRIPTS SHOW PERCENTAGE-POINT CHANGES FROM LATENT GOAL ONLY. BOLD MARKS THE BEST MEAN.

Setting	OGBench-				
	Push-T	Cube	Reacher	TwoRooms	Mean
$\lambda_s = 0$	90	94	86	94	91.0
$\lambda_s = 0.1$	92 ⁺²	98 ⁺⁴	86	98 ⁺⁴	93.5 ^{+2.5}
$\lambda_s = 0.3$	92 ⁺²	98 ⁺⁴	84 ⁻²	98 ⁺⁴	93.0 ⁺²
$\lambda_s = 1$	92 ⁺²	96 ⁺²	68 ⁻¹⁸	98 ⁺⁴	88.5 ^{-2.5}
$\lambda_s = 3$	90	98 ⁺⁴	68 ⁻¹⁸	100 ⁺⁶	89.0 ⁻²
$\lambda_s = 10$	94⁺⁴	100⁺⁶	92⁺⁶	100⁺⁶	96.5^{+5.5}

TABLE VII

TERMINAL ENERGY COMPOSITION. PAIRED 50-TRIAL PROTOCOL. TASK ENTRIES AND MEAN ARE SUCCESS RATES (%). MEAN IS UNWEIGHTED. SUPERSCRIPTS SHOW PERCENTAGE-POINT CHANGES FROM LATENT GOAL ONLY. BOLD MARKS THE BEST MEAN.

Setting	OGBench-				
	Push-T	Cube	Reacher	TwoRooms	Mean
Latent goal	90	94	86	94	91.0
Latent + damping	92 ⁺²	98 ⁺⁴	88 ⁺²	100 ⁺⁶	94.5 ^{+3.5}
Latent + terminal state	94⁺⁴	100⁺⁶	92⁺⁶	100⁺⁶	96.5^{+5.5}
All three terms	94 ⁺⁴	96 ⁺²	86	100 ⁺⁶	94.0 ⁺³

shows that the two auxiliary terms are not additive under this optimizer. Terminal-state matching penalizes both terminal predictions against the goal, while damping penalizes their separation regardless of goal error. The latter introduces a stationary-state preference that competes with goal matching when the decoder-predicted terminal descriptor is inaccurate. We therefore retain latent goal plus terminal-state matching and exclude damping energy from final LEAP.

E. Optimizer Efficiency

Table VIII compares optimizer choices under the final LEAP energy. Task columns and Mean are success rates. Mean superscripts give percentage-point changes from the L-BFGS/10 reference. The L-BFGS/10 row reports the final planner configuration. The remaining rows report the optimizer sweep.

Within the paired optimizer study, L-BFGS/10 gives the highest mean success at 96.5%. The other tested configurations range from 91.0% to 94.0%, so L-BFGS/10 remains the final design.

F. Qualitative Rollout Analysis

Fig. 5 analyzes paired rollouts for all four domains. The upper row for each task is LeWM+CEM and the lower row is LEAP under the same displayed target. Here, T denotes the primitive environment step. LEAP reaches the goal by $T = 23$ on OGBench-Cube and by $T = 28$ on Reacher, while LeWM+CEM stays away from the target throughout the displayed sequence. Push-T and TwoRooms instead show paired starts where both planners succeed and LEAP finishes sooner. LEAP requires 23 versus 25 steps on Push-T and 9 versus 24 steps on TwoRooms. For display times after a rollout terminates, the terminal frame is repeated. The status marks make the first successful displayed state explicit. Aggregate success rates across the evaluation schedule appear in Fig. 4.

TABLE VIII

OPTIMIZER ABLATION. PAIRED 50-TRIAL PROTOCOL. TASK ENTRIES AND MEAN ARE SUCCESS RATES (%). MEAN IS UNWEIGHTED. SUPERSCRIPTS SHOW PERCENTAGE-POINT CHANGES FROM L-BFGS/10. BOLD MARKS THE BEST MEAN.

Setting	OGBench-				
	Push-T	Cube	Reacher	TwoRooms	Mean
L-BFGS / 10 (ours)	94	100	92	100	96.5
Adam / 20	92	100	80	100	93.0 ^{-3.5}
Adam / 40	94	100	76	100	92.5 ⁻⁴
L-BFGS / 5	92	98	88	98	94.0 ^{-2.5}
L-BFGS / 20	92	96	78	98	91.0 ^{-5.5}

VI. DISCUSSION AND CONCLUSION

We present LEAP, a differentiable latent-energy planner for action refinement through a frozen world model. Its energy combines latent-goal agreement with decoder-predicted terminal-state matching, while post-optimization projection enforces admissible controls. This formulation defines a differentiable energy over action sequences evaluated through pretrained world-model rollouts without retraining the representation or dynamics model.

Across four manipulation and navigation domains, LEAP raises mean success from 77.5% with native CEM to 94.8% while adding 0.08 s of planning time per trial. Because the official LeWM encoder and predictor remain frozen, the improvement reflects the planning-system design rather than changes to the learned representation or dynamics model. This complete-system comparison does not isolate every planner component.

Limitations and Future Directions. LEAP inherits the frozen world model’s limitations on long-horizon and out-of-distribution rollouts. LEAP also uses a decoder trained to predict state descriptors from frozen LeWM embeddings and currently assumes a numerical goal descriptor y_g , which may not be available in uninstrumented domains. A promising extension is to replace y_g with a geometry-aware representation $G(g)$ inferred directly from goal images, such as object-centric keypoints or dense visual correspondences learned from unlabeled video. Future work could evaluate this visually grounded formulation under occlusion and domain shift, alongside broader physical and long-horizon evaluation.

REFERENCES

- [1] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 2555–2565.
- [2] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=S11OTC4tDS>
- [3] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, pp. 647–653, 2025.
- [4] N. Hansen, H. Su, and X. Wang, “TD-MPC2: Scalable, robust world models for continuous control,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [5] Y. LeCun, “A path towards autonomous machine intelligence,” *OpenReview*, 2022.

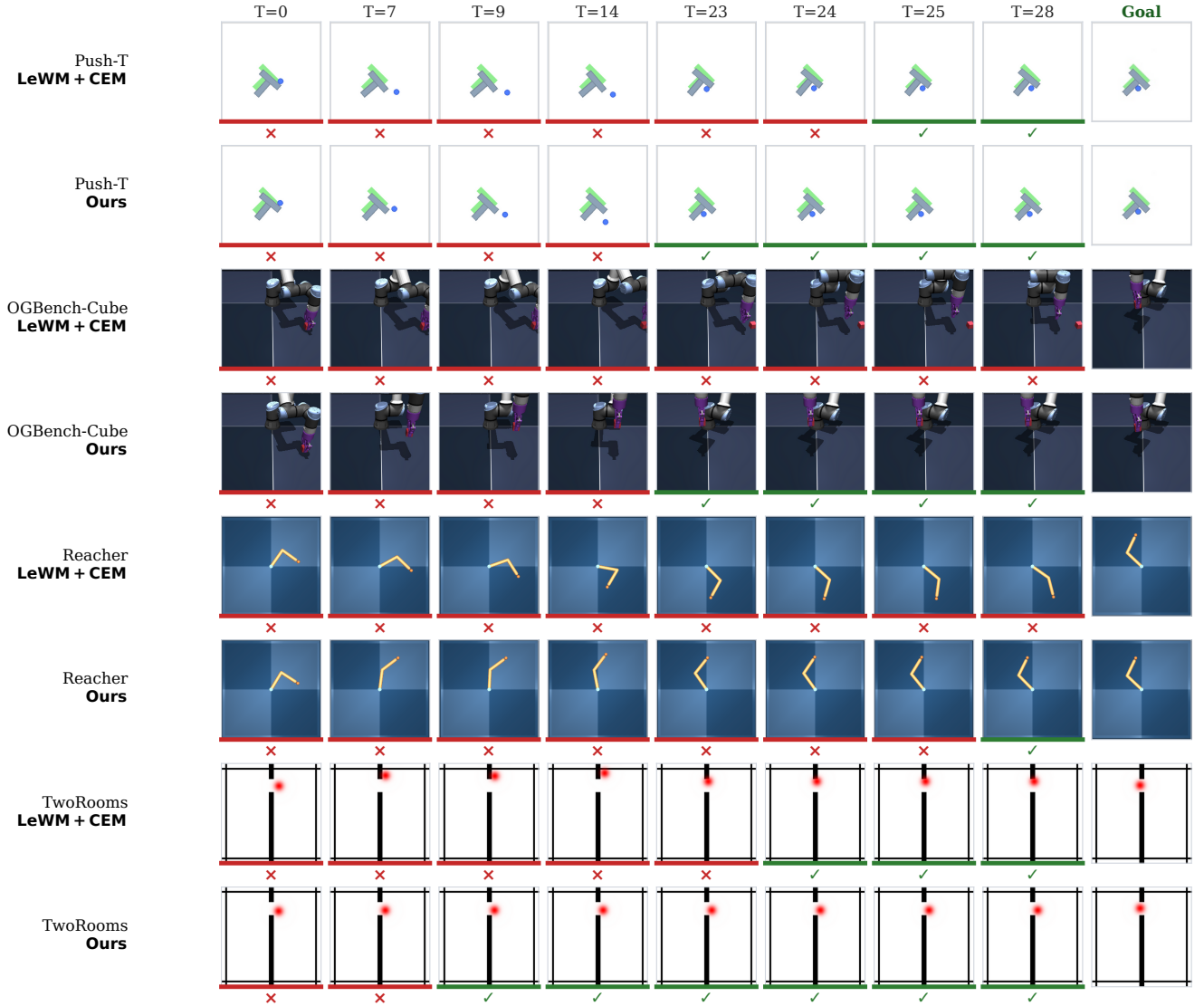


Fig. 5. Paired qualitative rollout comparison across four domains. Red crosses denote states before goal attainment, green check marks denote states at or after goal attainment, and the final column shows the goal.

- [6] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 15 619–15 629.
- [7] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas, “Revisiting feature prediction for learning visual representations from video,” *Transactions on Machine Learning Research*, 2024, featured Certification.
- [8] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, “DINO-WM: World models on pre-trained visual features enable zero-shot planning,” in *Forty-second International Conference on Machine Learning*, 2025.
- [9] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, “LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels,” *arXiv preprint arXiv:2603.19312*, 2026.
- [10] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, “Morel: Model-based offline reinforcement learning,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 21 810–21 823. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/f7efa4f864ae9b88d43527f4b14f750f-Paper.pdf
- [11] T. Yu, G. Thomas, L. Yu, S. Ermon, J. Zou, S. Levine, C. Finn, and T. Ma, “Mopo: Model-based offline policy optimization,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 14 129–14 142. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/a322852ce0df73e204b7e67cbbef0d0a-Paper.pdf
- [12] A. Kumar, J. Fu, G. Tucker, and S. Levine, *Stabilizing off-policy Q-learning via bootstrapping error reduction*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [13] R. Rubinstein, “The cross-entropy method for combinatorial and continuous optimization,” *Methodology and Computing in Applied Probability*, vol. 1, no. 2, pp. 127–190, 1999.
- [14] N. A. Hansen, H. Su, and X. Wang, “Temporal difference learning for model predictive control,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 8387–8406.
- [15] K. Chua, R. Calandra, R. McAllister, and S. Levine, “Deep reinforcement learning in a handful of trials using probabilistic dynamics models,” in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS’18. Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 4759–4770.
- [16] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine, “Visual foresight: Model-based deep reinforcement learning for vision-based robotic control,” *arXiv preprint arXiv:1812.00568*, 2018.

- [17] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable mpc for end-to-end planning and control," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 8299–8310.
- [18] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *5th Annual Conference on Robot Learning*, 2021.
- [19] J. Urain, A. Li, P. Liu, C. D'Eramo, and J. Peters, "Composable energy policies for reactive motion generation and reinforcement learning," *Int. J. Rob. Res.*, vol. 42, no. 10, pp. 827–858, Sep. 2023. [Online]. Available: <https://doi.org/10.1177/02783649231179499>
- [20] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Rob. Res.*, vol. 44, no. 10-11, pp. 1684–1704, Sep. 2025. [Online]. Available: <https://doi.org/10.1177/02783649241273668>
- [21] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, pp. 503–528, 1989.